Proyecto #2 – INFO1157: Sistemas Inteligentes

Sistema Distribuido de Monitoreo Ambiental con MQTT, Matplotlib y Telegram

Docente: Dr. (c) Alberto Caro | Documento Técnico de Arquitectura y Código

1. Resumen General del Proyecto

El presente proyecto implementa una solución de telemetría y monitoreo ambiental distribuido basado en el patrón arquitectónico **Publicador/Suscriptor (Pub/Sub)** mediante el protocolo **MQTT** sobre un broker local **Mosquitto** (127.0.0.1:1883), integrando generación automática de reportes gráficos y su posterior despacho a la plataforma de mensajería **Telegram**.

La arquitectura del sistema está conformada por los siguientes módulos interactuantes:

Módulo	Componente	Descripción y Responsabilidad
Publicador	publicador.py	Genera valores sintéticos aleatorios para material particulado (MP01, MP25, MP10) y condiciones atmosféricas (Temperatura, Humedad Relativa), publicándolos periódicamente cada 5 segundos alternando entre los tópicos DATA/MP y DATA/MA.
Broker MQTT	Mosquitto (127.0.0.1:1883)	Servidor central de mensajería que desacopla la emisión de la recepción, enrutando los mensajes a todos los clientes suscritos sin mantener estado persistente.
Suscriptores	suscriptor_mqtt_1..4.py	Cuatro procesos independientes que se ejecutan en paralelo. Cada uno se suscribe simultáneamente a DATA/MP y DATA/MA, parsea los strings numéricos y los acumula en 5 listas separadas hasta completar exactamente 10 muestras históricas sincronizadas.
Reporte Gráfico	Matplotlib (Backend Agg)	Al alcanzar las 10 muestras, se dispara la función de graficado que crea una figura con 5 subgráficos apilados verticalmente (réplica exacta de la Figura 3 de la pauta) y la guarda localmente como PNG.
Bot Telegram	Telepot (API Bots)	Cada uno de los 4 suscriptores cuenta con un Token de Bot diferente de Telegram. Al generarse el archivo gráfico, se despacha automáticamente mediante sendPhoto() al Chat ID del usuario y acto seguido se vacían las listas para reiniciar la ventana de captura.

2. Dónde y Cómo se Usan las Librerías

A continuación se especifica la justificación y punto de invocación de cada dependencia externa dentro del código fuente:

Librería	Uso en el Código	Justificación Técnica

paho.mqtt.client	`import paho.mqtt.client as mqtt`
	`mqtt.Client(...)`
	`cliente.connect(...)`
	`client.subscribe(...)`
	`cliente.loop_forever()`
		Permite la comunicación orientada a eventos con el Broker Mosquitto mediante sockets TCP. Implementa la arquitectura Callback API v2 para gestionar la reconexión, el handshake y la recepción asíncrona de mensajes.

matplotlib / pyplot	`matplotlib.use('Agg')`
	`plt.subplots(5, 1, ...)`
	`ax.plot(...)`
	`plt.savefig(...)`
	`plt.close(fig)`
		Construcción del dashboard de visualización ambiental. El uso explícito de matplotlib.use('Agg') desactiva la capa gráfica del sistema operativo, permitiendo renderizar y exportar imágenes PNG sin bloquear los hilos de ejecución en procesos concurrentes.

telepot	`import telepot`
	`bot = telepot.Bot(TOKEN)`
	`bot.sendPhoto(...)`
		Cliente HTTP para la API REST de Telegram. Facilita la transmisión directa de archivos binarios (fotos) con pie de mensaje hacia el chat privado sin requerir configuración compleja de webhooks.

os	`import os`	Manejo de rutas del sistema operativo y verificación de archivos locales.

3. Explicación Bloque por Bloque

El código de cada suscriptor está estructurado en 8 bloques lógicos claramente delimitados:

Bloque 1: Importación de Dependencias (Líneas 1-6)

Carga en el espacio de nombres las librerías necesarias. Destaca la llamada inmediata a matplotlib.use('Agg') antes de importar pyplot, lo que garantiza que Matplotlib opere en modo headless (sin interfaz gráfica), evitando errores por concurrencia al ejecutarse cuatro instancias en simultáneo.

Bloque 2: Constantes y Parámetros de Configuración (Líneas 8-20)

Agrupa todas las variables modificables: Token del bot de Telegram, Chat ID, client_id de MQTT (único para cada suscriptor: suscriptor_01 a suscriptor_04), dirección IP del broker (127.0.0.1), puerto (1883), nombres de los tópicos (DATA/MP y DATA/MA), umbral de muestras (10) y nombre del archivo PNG destino.

Bloque 3: Buffers de Almacenamiento en Memoria (Líneas 22-26)

Inicializa las cinco listas globales en memoria (lista_mp01, lista_mp25, lista_mp10, lista_tempe, lista_hr). Estas listas funcionan como acumuladores FIFO que registran secuencialmente las lecturas que llegan desde los mensajes MQTT.

Bloque 4: Validación de Acumulación (Líneas 28-32)

Función todas_las_listas_completas(). Aplica la función nativa all() sobre un generador que verifica si cada una de las 5 listas posee una longitud mayor o igual a 10. Solo devuelve True cuando todas las variables cuentan con datos suficientes para graficar.

Bloque 5: Renderizado y Despacho del Reporte (Líneas 34-73)

Función graficar_y_enviar(). Parametriza las 5 curvas con sus colores (azul, verde, rojo), escalas (0-50 para material particulado y temperatura; 50-90 para humedad), grilla punteada y leyendas a la derecha con borde. Guarda la imagen en disco, la envía a Telegram por Telepot dentro de un bloque try-except y, tras el envío exitoso, vacía las 5 listas con .clear() para reiniciar la recolección.

Bloque 6: Callback de Conexión MQTT (Líneas 75-81)

Función on_connect(). Cumple con la firma Callback API v2 de Paho (client, userdata, flags, reason_code, properties). Si reason_code == 0, suscribe al cliente de forma simultánea a DATA/MP y DATA/MA con QoS 0.

Bloque 7: Callback de Procesamiento de Mensajes (Líneas 83-111)

Función on_message(). Al recibir un paquete MQTT, decodifica el payload a UTF-8 y aplica .strip() para limpiar saltos de línea. Discrimina por tópico, separa por comas con .split(',') y castea los valores a float. Si se alcanzan 10 muestras completas, dispara graficar_y_enviar().

Bloque 8: Ciclo Principal y Manejo de Señales (Líneas 113-128)

Función main(). Instancia el cliente con su client_id único, enlaza callbacks, ejecuta cliente.connect() e ingresa a cliente.loop_forever(), capturando KeyboardInterrupt (Ctrl+C) para desconectar el socket limpiamente.

4. Explicación Detallada Línea por Línea

A continuación se presenta el desglose analítico de cada línea de código del suscriptor:

Línea(s)	Fragmento de Código	Explicación Técnica
1	`import paho.mqtt.client as mqtt`	Importa el cliente de Paho MQTT y lo asocia al alias estándar 'mqtt'.
2	`import matplotlib`	Carga el paquete principal de Matplotlib.
3	`matplotlib.use('Agg')`	Establece el backend no interactivo 'Agg' para generar gráficos sin ventanas de GUI.
4	`import matplotlib.pyplot as plt`	Importa la interfaz procedural pyplot bajo el alias común 'plt'.
5	`import telepot`	Importa el módulo Telepot para controlar el bot de Telegram vía HTTP API.
6	`import os`	Importa utilidades del sistema operativo para manejo de rutas y archivos.
8	`TELEGRAM_TOKEN = '...'`	Token de autenticación emitido por @BotFather para este bot específico.
9	`CHAT_ID = '8483080265'`	Identificador numérico del chat destinatario donde el bot enviará la imagen.
10	`MQTT_CLIENT_ID = 'suscriptor_01'`	ID único de cliente MQTT (evita colisiones con las otras 3 instancias).
12	`BROKER_HOST = '127.0.0.1'`	Dirección IP local donde escucha el broker Mosquitto.
13	`BROKER_PORT = 1883`	Puerto estándar TCP para conexiones MQTT sin cifrado.
15	`TOPICO_MP = 'DATA/MP'`	Canal MQTT destinado a las mediciones de material particulado.
16	`TOPICO_MA = 'DATA/MA'`	Canal MQTT destinado a las mediciones de medio ambiente (temperatura y humedad).
18	`MUESTRAS_PARA_GRAFICAR = 10`	Cantidad de registros históricos requeridos por variable para armar el gráfico.
19	`NOMBRE_IMAGEN = 'grafico_bot_1.png'`	Nombre del archivo de imagen PNG que generará esta instancia.
21	`lista_mp01 = []`	Buffer dinámico para almacenar las muestras de MP 1.0 µm.
22	`lista_mp25 = []`	Buffer dinámico para almacenar las muestras de MP 2.5 µm.
23	`lista_mp10 = []`	Buffer dinámico para almacenar las muestras de MP 10.0 µm.
24	`lista_tempe = []`	Buffer dinámico para almacenar las muestras de Temperatura (°C).
25	`lista_hr = []`	Buffer dinámico para almacenar las muestras de Humedad Relativa (%).
27	`def todas_las_listas_completas():`	Define la función que verifica si se reunieron los datos requeridos.
28-31	`return all(len(lst) >= 10 for lst in (...))`	Evalúa si cada una de las 5 listas tiene al menos 10 muestras acumuladas.
33	`def graficar_y_enviar():`	Define la función que genera la figura, la envía y resetea las listas.
34-40	`config_plots = [...]`	Tupla con la configuración de cada variable: datos[:10], nombre, unidad, color, límites y marcas Y.
42	`fig, ejes = plt.subplots(5, 1, ...)`	Crea la figura con 5 subplots apilados compartiendo el eje X (sharex=True).
43	`fig.suptitle('Medicion MP1, MP2.5, ...')`	Añade el título principal centrado superior exigido en la Figura 3.
45	`muestras_eje = list(range(1, 11))`	Genera la secuencia horizontal de índices [1, 2, ..., 10] para las muestras.
47	`for ax, (valores, ...) in zip(ejes, ...):`	Itera simultáneamente sobre cada eje y sus parámetros correspondientes.

48	`ax.plot(muestras_eje, valores, color=...)`	Grafica la curva de datos con el color y grosor especificado.
49	`ax.set_ylabel(ylabel, fontsize=10)`	Establece el rótulo del eje Y con la unidad física (ug/m3, Tempe, HR(%)).
50	`ax.set_ylim(ylim)`	Fija los rangos mínimo y máximo vertical (0-50 o 50-90).
51	`ax.set_yticks(yticks)`	Configura las marcas numéricas de escala vertical exactamente como la Figura 3.
52	`ax.set_xlim(1, 13)`	Establece el rango horizontal entre 1 y 13 para dar holgura al trazo.
53	`ax.grid(True, linestyle=':', ...)`	Dibuja la grilla de líneas punteadas grises.
54	`ax.legend(loc='center right', ...)`	Ubica el cuadro de leyenda con borde negro y fondo blanco a la derecha.
55	`ax.tick_params(direction='in', ...)`	Orienta las marcas de graduación hacia el interior en los 4 bordes.
57	`ejes[-1].set_xlabel('10 Samples Datas...')`	Coloca la etiqueta inferior '10 Samples Datas...' únicamente en el último gráfico.
58	`ejes[-1].set_xticks([2, 4, 6, 8, 10, 12])`	Configura las marcas numéricas pares del eje horizontal.
60	`plt.subplots_adjust(...)`	Ajusta márgenes y espaciado vertical entre subgráficos para evitar solapamientos.
61	`plt.savefig(NOMBRE_IMAGEN, dpi=150)`	Exporta la figura completa a archivo PNG en disco a 150 DPI.
62	`plt.close(fig)`	Cierra la figura y libera la memoria RAM asignada al objeto gráfico.
63	`print(f'[INFO] Gráfico guardado...')`	Muestra en consola la confirmación de guardado local.
65-68	`bot = telepot.Bot(...); bot.sendPhoto(...)`	Abre la imagen en binario y la envía al chat de Telegram con pie descriptivo.
69-71	`try-except Exception as e`	Manejo de excepciones para que errores de red o Telegram no tiren el proceso.
73-77	`lista_mp01.clear() ... lista_hr.clear()`	Vacía las 5 listas para recibir el próximo bloque de 10 muestras sin parar el cliente.
80	`def on_connect(client, userdata, ...):`	Callback ejecutado cuando Mosquitto responde a la conexión.
81-84	`if reason_code == 0: client.subscribe(...)`	Al conectarse con éxito, se suscribe en paralelo a 'DATA/MP' y 'DATA/MA'.
88	`def on_message(client, userdata, msg):`	Callback ejecutado cada vez que arriba una publicación en un tópico suscrito.
89-90	`topico = msg.topic; payload = ...strip()`	Obtiene el nombre del tópico y decodifica el payload limpiando caracteres ocultos.
93-101	`if topico == TOPICO_MP: ...`	Parsea los 3 valores de MP separados por comas, los castea a float y los anexa a sus listas.
105-111	`elif topico == TOPICO_MA: ...`	Parsea temperatura y humedad, los convierte a float y los anexa a sus listas.
115-117	`except (ValueError, IndexError):`	Descarta mensajes malformados sin abortar la ejecución.
119-121	`if todas_las_listas_completas(): graficar...`	Dispara el reporte cuando todas las listas acumulan al menos 10 datos.
123-129	`def main(): cliente.connect(...)`	Instancia el cliente con API v2, asigna callbacks y conecta al broker.
131-136	`cliente.loop_forever(); except Keyboard...`	Bucle bloqueante de escucha continua con captura limpia de Ctrl+C.

138-139	`if __name__ == '__main__': main()`	Garantiza la ejecución del script solo si es invocado directamente.

5. Guía de Ejecución y Pruebas del Repositorio

Para ejecutar el sistema completo en una máquina de evaluación o laboratorio, seguir los siguientes pasos secuenciales:

Paso 1: Instalación de Dependencias

Ejecutar en la terminal de comandos:
py -m pip install paho-mqtt matplotlib telepot

Paso 2: Iniciar el Broker MQTT

Asegurarse de que el servicio de **Mosquitto** esté activo en el puerto 1883 de 127.0.0.1 (o iniciar el binario mosquitto.exe -v).

Paso 3: Lanzar los 4 Clientes Suscriptores en Paralelo

Abrir 4 consolas independientes y ejecutar un suscriptor en cada una:
Consola 1: py suscriptor_mqtt_1.py
Consola 2: py suscriptor_mqtt_2.py
Consola 3: py suscriptor_mqtt_3.py
Consola 4: py suscriptor_mqtt_4.py

Paso 4: Ejecutar el Publicador

En una 5ª consola, iniciar el script del publicador oficial:
py publicador.py

Paso 5: Verificación en Telegram

A medida que el publicador emita datos, cada suscriptor procesará las tramas. Al llegar a la muestra 10, cada bot enviará de forma automática su gráfica respectiva al chat de Telegram con el diseño oficial de la Figura 3.